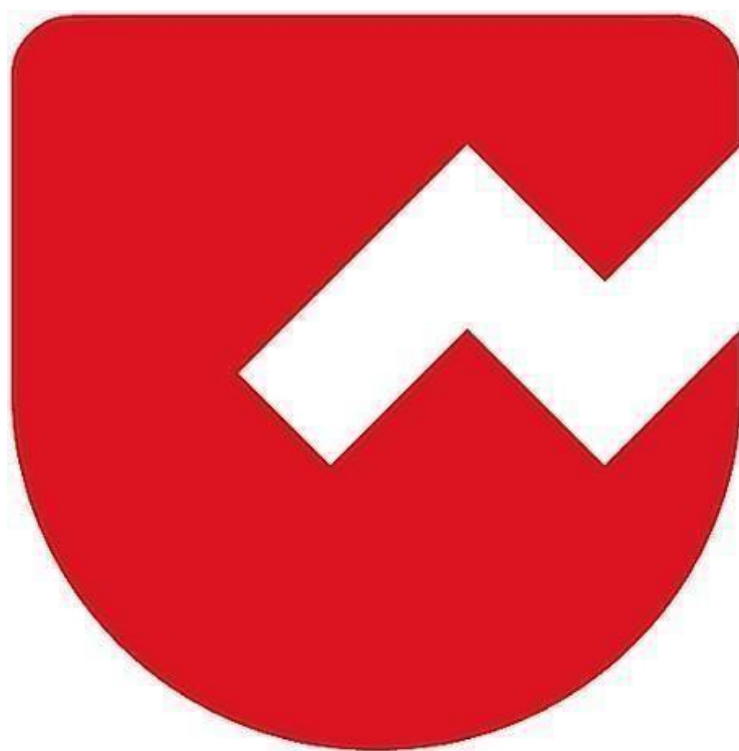


Zoho Schools for Graduate Studies

Notes



Inner Class

An Inner Class in Java is a class defined inside another class. The class inside is called the inner class, and the outer one is called the outer class.

Example

```
class Outer {  
  
    class Inner {  
        // inner class code  
    }  
}
```

Key Points:

- Inner class is tightly connected to the outer class
- It can access:
 - Outer class variables
 - Outer class methods
- Even private members of outer class
- Inner class cannot exist independently
- Object of inner class depends on object of outer class

- Inner class can access private data of outer class, but outside classes cannot access inner class directly.

USE CASES

- Inner classes are used in Java for better design, security, and readability.
- Inner class can access private data of outer class, but outside classes cannot access inner class directly.(ENCAPULATION)
- Improves Security
- Inner class is hidden
- Cannot be accessed without outer class
- Reduces misuse of helper classes

Types of inner class

- Member Inner Class
- Static Inner Class
- Local Inner Class
- Anonymous Inner Class

Member Inner Class

- A class defined inside another class
- It is non-static
- Needs outer class object to create inner class object
- Can access all members of outer class (including private)
- Used when inner class is closely related to outer class

Example

```
class Outer {  
    class Inner {  
        void show() {  
            System.out.println("Hello Inner");  
        }  
    }  
}
```

```
public class Test {  
    public static void main(String[] args) {  
        Outer o = new Outer();           // outer object  
        Outer.Inner i = o.new Inner();    // inner object  
        i.show();  
    }  
}
```

```

public class Test {
    public static void main(String[] args) {
        Outer.Inner i = new Outer().new Inner();
        i.show();
    }
}

```

Example 2

```

class Outer {
    void display() {
        class Inner {
            void msg() {
                System.out.println("Local Inner Class");
            }
        }
        Inner i = new Inner();
        i.msg();
    }
}

```

```

public class Test {
    public static void main(String[] args) {
        Outer o = new Outer();
        o.display();
    }
}

```

Static Inner Class

- A class defined inside another class using static
- Does not need outer class object
- Can access only static members of outer class
- Behaves like a normal class but grouped inside another class
- Used when inner logic is related but independent

```
class Outer {  
    static class Inner {  
        void show() {  
            System.out.println("Static Inner");  
        }  
    }  
}
```

```
public class Test {  
    public static void main(String[] args) {  
        Outer.Inner i = new Outer.Inner(); // no outer object  
        i.show();  
    }  
}
```

Anonymous Inner Class

- Inner class without a name

- Created for one-time use
- Mostly used with interfaces or abstract classes
- Reduces code length
- Common in event handling and threads

```
interface A {  
    void show();  
}  
  
public class Test {  
    public static void main(String[] args) {  
        A obj = new A() {  
            public void show() {  
                System.out.println("Anonymous Inner Class");  
            }  
        };  
        obj.show();  
    }  
}
```

Lazy Loading in Inner Classes

- Outer class loads first
- Inner class loads only when it is used
- If inner class is never used → it is never loaded

- Saves memory
- Faster program startup
- Loads only required classes
- Important for large applications

Example

```
class Outer {  
  
    static {  
        System.out.println("Outer class loaded");  
    }  
  
    class Inner {  
        static {  
            System.out.println("Inner class loaded");  
        }  
  
        void show() {  
            System.out.println("Inner method called");  
        }  
    }  
}
```

Main Class (Case 1: Inner NOT used)

```
public class Test {  
    public static void main(String[] args) {  
        Outer o = new Outer();  
        System.out.println("Main ends");  
    }  
}
```



```
}  
}
```

Output

Outer class loaded
Main ends

Inner class NOT loaded
Because it was never used

Main Class (Case 2: Inner USED)

```
public class Test {  
    public static void main(String[] args) {  
        Outer o = new Outer();  
        Outer.Inner i = o.new Inner();  
        i.show();  
    }  
}
```

Output

Outer class loaded
Inner class loaded
Inner method called

Inner class loads only when object is created
This is Lazy Loading

Compilation and runtime

At Compilation Time

Java compiler creates:

- Outer.class
- Outer\$Inner.class

Both .class files are created

At Runtime (JVM Execution)

- JVM does NOT load all .class files

JVM loads:

- Only classes that are used
- Inner class .class file exists but JVM loads it only when required